

Get Wallet Balance

Retrieve your current wallet balance to check available funds for placing orders.

Endpoint

```
GET /v1/delivery-partners/{dpID}/wallet/balance
```

Headers

Header	Type	Description	Required
x-client-id	String	Your clientID	☒ Yes
x-client-secret	String	Your clientSecret	☒ Yes

Path Parameters

Parameter	Type	Description	Required
dpID	String	Your delivery partner ID	☒ Yes

Example Request

```
curl --location 'https://stage-platform-exlr8.exlr8now.com/v1/delivery-partners/YOUR_DP_ID/wallet/balance' \
```

```
--header 'x-client-id: YOUR_CLIENT_ID' \  
--header 'x-client-secret: YOUR_CLIENT_SECRET'
```

Response

Successful Response

```
{  
  "balance": 90000,  
  "currency": "INR"  
}
```

Response Fields

Field	Type	Description
balance	Number	Current wallet balance
currency	String	Currency code (typically "INR")

Balance Monitoring

Real-time Balance Checks

```
// Example: Check balance before placing order  
async function checkBalanceBeforeOrder(orderAmount) {  
  try {  
    const response = await fetch(  
      `${baseUrl}/v1/delivery-partners/${dpID}/wallet/balance`,  
      {  

```

```

        headers: {
          'x-client-id': clientId,
          'x-client-secret': clientSecret
        }
      }
    );

    const walletData = await response.json();

    if (walletData.balance >= orderAmount) {
      console.log('Sufficient balance available');
      return true;
    } else {
      console.warn(`Insufficient balance. Available:
${walletData.balance}, Required: ${orderAmount}`);
      return false;
    }
  } catch (error) {
    console.error('Error checking wallet balance:', error);
    throw error;
  }
}

```

Balance Alerts

```

// Example: Set up balance monitoring
async function monitorBalance(minimumThreshold = 10000) {
  const walletData = await getWalletBalance();

  if (walletData.balance < minimumThreshold) {
    // Send alert to admin
    await sendLowBalanceAlert({
      currentBalance: walletData.balance,
      threshold: minimumThreshold,
      currency: walletData.currency
    });
  }
}

```

```
}
```

Error Responses

Unauthorized Access

```
{  
  "error": "unauthenticated",  
  "errCode": "UNAUTHORIZED"  
}
```

Invalid DP ID

```
{  
  "error": "forbidden: param: admin user does not have access to DP:  
INVALID_DP_ID",  
  "errCode": "FORBIDDEN"  
}
```

Use Cases

1. Pre-order Validation

Check balance before allowing customers to place orders.

2. Dashboard Display

Show current balance on admin dashboards.

3. Low Balance Alerts

Monitor balance and alert when funds are running low.

4. Financial Reporting

Include balance information in financial reports.

Best Practices

1. **Check Before Orders:** Always verify sufficient balance before placing orders
2. **Cache Wisely:** Balance changes frequently, so cache for short periods only
3. **Handle Errors:** Implement proper error handling for balance checks
4. **Monitor Regularly:** Set up automated balance monitoring
5. **Alert Thresholds:** Configure low balance alerts for your business needs

Integration Patterns

Pre-order Balance Check

```
// Recommended pattern: Check balance before order
async function placeOrderWithBalanceCheck(orderData) {
  // 1. Calculate order total
  const orderTotal = calculateOrderTotal(orderData);

  // 2. Check wallet balance
  const hasBalance = await checkBalanceBeforeOrder(orderTotal);

  if (!hasBalance) {
    throw new Error('Insufficient wallet balance');
  }

  // 3. Place order
  return await placeOrder(orderData);
}
```

```
}
```

Balance Monitoring Service

```
// Example: Background balance monitoring
class WalletMonitor {
  constructor(dpID, clientId, clientSecret, thresholds) {
    this.dpID = dpID;
    this.clientId = clientId;
    this.clientSecret = clientSecret;
    this.thresholds = thresholds;
  }

  async startMonitoring(intervalMinutes = 30) {
    setInterval(async () => {
      try {
        const balance = await this.getBalance();
        await this.checkThresholds(balance);
      } catch (error) {
        console.error('Balance monitoring error:', error);
      }
    }, intervalMinutes * 60 * 1000);
  }

  async getBalance() {
    const response = await fetch(
      `${baseUrl}/v1/delivery-partners/${this.dpID}/wallet/balance`,
      {
        headers: {
          'x-client-id': this.clientId,
          'x-client-secret': this.clientSecret
        }
      }
    );

    return await response.json();
  }
}
```

```
async checkThresholds(walletData) {  
  const { balance } = walletData;  
  
  if (balance < this.thresholds.critical) {  
    await this.sendAlert('CRITICAL', balance);  
  } else if (balance < this.thresholds.warning) {  
    await this.sendAlert('WARNING', balance);  
  }  
}
```

Get Wallet Transaction History

Retrieve your wallet transaction history with optional filtering by transaction type.

Endpoint

GET /v1/delivery-partners/{dpID}/wallet/transactions

Headers

Header	Type	Description	Required
x-client-id	String	Your clientID	☒ Yes

x-client-secret	String	Your clientSecret	☒ Yes
-----------------	--------	-------------------	---

Path Parameters

Parameter	Type	Description	Required
dpID	String	Your delivery partner ID	☒ Yes

Query Parameters

Parameter	Type	Description	Required	Options
txnType	String	Filter by transaction type	☒ No	CREDIT, DEBIT
nextCursor	String	Pagination cursor	☒ No	-
limit	Integer	Records per page	☒ No	Default: 15, Max: 100

Transaction Types

Type	Description
CREDIT	Money is credited into the wallet
DEBIT	Money is debited from the wallet

Example Requests

Get All Transactions

```
curl --location 'https://stage-platform-exlr8.exlr8now.com/v1/delivery-partners/YOUR_DP_ID/wallet/transactions' \
  --header 'x-client-id: YOUR_CLIENT_ID' \
  --header 'x-client-secret: YOUR_CLIENT_SECRET'
```

Get Only Debit Transactions

```
curl --location 'https://stage-platform-exlr8.exlr8now.com/v1/delivery-partners/YOUR_DP_ID/wallet/transactions' \
  --data-urlencode 'txnType=DEBIT' \
  --header 'x-client-id: YOUR_CLIENT_ID' \
  --header 'x-client-secret: YOUR_CLIENT_SECRET'
```

Get Transactions with Pagination

```
curl --location 'https://stage-platform-exlr8.exlr8now.com/v1/delivery-partners/YOUR_DP_ID/wallet/transactions' \
  --data-urlencode 'limit={limit}' \
  --data-urlencode 'nextCursor={nextCursor}' \
  --header 'x-client-id: YOUR_CLIENT_ID' \
  --header 'x-client-secret: YOUR_CLIENT_SECRET'
```

Response

Successful Response

```
{
  "transactions": [
    {
      "txnID": "615261b6-caf9-42e0-aa7b-4becaf54a289-1755583153343128244",
      "dpID": "615261b6-caf9-42e0-aa7b-4becaf54a289",
      "txnType": "DEBIT",
      "currency": "INR",
      "amount": 95,
      "balanceBefore": 71199.66,
      "balanceAfter": 71104.66,
      "sourceSystem": "EXLR8_B2B_STORE",
      "activity": "PURCHASE",
      "referenceID": "ORDIN190820258b9c48b",
      "referenceType": "PURCHASE",
      "metadata": {
        "comment": "System Transaction: Money DEBITED against Order"
      },
      "createdAt": "2025-08-19T05:59:13.343Z",
      "updatedAt": "2025-08-19T05:59:13.343Z"
    }
  ],
  "paginationInfo": {
    "nextCursor": "68a15872f1ed13bfd892c123",
    "hasMore": true
  }
}
```

Response Fields

Transaction Fields

Field	Type	Description
txnID	String	Unique transaction identifier
dpID	String	Your delivery partner ID
txnType	String	Transaction type (CREDIT/DEBIT)
currency	String	Currency code
amount	Number	Transaction amount
balanceBefore	Number	Wallet balance before transaction
balanceAfter	Number	Wallet balance after transaction
sourceSystem	String	System that initiated the transaction
activity	String	Activity type (see Activity Types below)
referenceID	String	Reference to related entity (Order ID, UTR, etc.)
referenceType	String	Type of reference
metadata	Object	Additional transaction details
createdAt	String	Transaction timestamp
updatedAt	String	Last update timestamp

Transaction Activity Types

Activity	Description	Reference Type
TOPUP	Wallet topped up manually	UTR (bank reference)
PURCHASE	Purchase made from wallet	Order ID
REFUND	Money refunded against an order	Order ID
ADJUST	Manual wallet adjustment	Transaction ID

Filtering Examples

Get Recent Purchases

```
// Example: Get recent purchase transactions
async function getRecentPurchases(limit = 50) {
  const response = await fetch(
    `${baseUrl}/v1/delivery-
partners/${dpID}/wallet/transactions?txnType=DEBIT&limit=${limit}`,
    {
      headers: {
        'x-client-id': clientId,
        'x-client-secret': clientSecret
      }
    }
  );

  const data = await response.json();

  // Filter only purchase activities
  return data.transactions.filter(txn => txn.activity === 'PURCHASE');
```

```
}
```

Get Top-ups Only

```
// Example: Get wallet top-up history
async function getTopupHistory() {
  const response = await fetch(
    `${baseUrl}/v1/delivery-
partners/${dpID}/wallet/transactions?txnType=CREDIT`,
    {
      headers: {
        'x-client-id': clientId,
        'x-client-secret': clientSecret
      }
    }
  );

  const data = await response.json();

  // Filter only top-up activities
  return data.transactions.filter(txn => txn.activity === 'TOPUP');
}
```

Error Responses

Unauthorized Access

```
{
  "error": "unauthenticated",
  "errCode": "UNAUTHORIZED"
}
```

Invalid DP ID

```
{
  "error": "forbidden: param: admin user does not have access to DP: INVALID_DP_ID",
  "errCode": "FORBIDDEN"
}
```

Invalid Transaction Type

```
{
  "error": "Invalid transaction type",
  "errCode": "BAD_REQUEST"
}
```

Use Cases

1. Financial Reconciliation

Match transactions with your internal accounting systems.

2. Audit Trail

Maintain complete audit trails for compliance.

3. Spending Analysis

Analyze spending patterns and optimize purchasing.

4. Customer Support

Provide transaction details for customer inquiries.

Best Practices

1. **Regular Reconciliation:** Regularly reconcile transactions with your records
2. **Filter Appropriately:** Use transaction type filters to reduce data transfer
3. **Handle Pagination:** Implement proper pagination for large transaction histories
4. **Store References:** Map transaction references to your internal systems
5. **Monitor Patterns:** Watch for unusual transaction patterns

Integration Example

```
// Example: Complete transaction monitoring
class TransactionMonitor {
  async getTransactionSummary(days = 30) {
    const transactions = await this.getTransactionsForPeriod(days);

    const summary = {
      totalCredits: 0,
      totalDebits: 0,
      purchaseCount: 0,
      refundCount: 0,
      topupCount: 0
    };

    transactions.forEach(txn => {
      if (txn.txnType === 'CREDIT') {
        summary.totalCredits += txn.amount;
        if (txn.activity === 'TOPUP') summary.topupCount++;
        if (txn.activity === 'REFUND') summary.refundCount++;
      } else if (txn.txnType === 'DEBIT') {
        summary.totalDebits += txn.amount;
        if (txn.activity === 'PURCHASE') summary.purchaseCount++;
      }
    });
  }
}
```

```

    }
  });

  return summary;
}

async reconcileWithOrders() {
  const purchases = await this.getPurchaseTransactions();
  const orders = await this.getOrders();

  // Match transactions with orders
  const reconciliation = purchases.map(txn => {
    const order = orders.find(o => o.orderID === txn.referenceID);
    return {
      transaction: txn,
      order: order,
      matched: !!order,
      discrepancy: order ? Math.abs(order.totalAmount -
txn.amount) : null
    };
  });

  return reconciliation;
}
}

```

Get Available Products

Retrieve a list of products and variants available for ordering, with pagination support.

Endpoint

```
GET /v1/products/delivery-partners/{dpID}
```

Headers

Header	Type	Description	Required
x-client-id	String	Your clientID	☒ Yes
x-client-secret	String	Your clientSecret	☒ Yes

Path Parameters

Parameter	Type	Description	Required
dpID	String	Your delivery partner ID	☒ Yes

Query Parameters

Parameter	Type	Description	Required	Default
nextCursor	String	Pagination cursor for next page	☒ No	-
limit	Integer	Number of records per page	☒ No	15

PAGINATION LIMITS

- **Default limit:** 15 records per page
- **Maximum limit:** 100 records per page

Example Request

Get All Products (First Page)

```
curl --location 'https://stage-platform-exlr8.exlr8now.com/v1/products/delivery-partners/YOUR_DP_ID' \
--header 'x-client-id: YOUR_CLIENT_ID' \
--header 'x-client-secret: YOUR_CLIENT_SECRET'
```

Get Products with Pagination

```
curl --location 'https://stage-platform-exlr8.exlr8now.com/v1/products/delivery-partners/YOUR_DP_ID' \
--data-urlencode 'limit={limit}' \
--data-urlencode 'nextCursor={nextCursor}' \
--header 'x-client-id: YOUR_CLIENT_ID' \
--header 'x-client-secret: YOUR_CLIENT_SECRET'
```

Response

Successful Response

```
{
  "products": [
    {
      "productID": "PROD-cd250a2a-2b27-40e0",
      "attachments": [
        "https://storage.googleapis.com/exlr8-assets/0e6ffdd-67e7-41f1-bf87-12adc597d4cb/images/products/default_voucher.jpg"
      ],
      "categories": [
        {
```

```
        "categoryID": "CAT-e0bc01c8-a9b2-4d3a",
        "categoryName": "Gaming"
    }
],
    "descriptionText": "Get this voucher for instant savings on your next purchase. Simply redeem it at the time of checkout to apply the discount. It's the perfect way to make your money go further",
    "productDisplayName": "TEST_PRODUCT_1_DP_APIS",
    "productName": "TEST_PRODUCT_1_DP_APIS",
    "redemptionInstructions": "To redeem your voucher, follow these steps:\n\nClick on the unique redemption URL provided in your email or on the voucher page.\n\nAdd your desired products to the cart on the partner's website.\n\nThe discount will be automatically applied at checkout, or you may need to enter a unique code found on your voucher.\n\nComplete your purchase and enjoy your savings!",
    "termsAndConditions": "This voucher is valid for one-time use only. 2. It cannot be combined with any other offers, discounts, or promotions. 3. This voucher is non-refundable and cannot be exchanged for cash. 4. The voucher is valid until its stated expiration date. 5. Lost, stolen, or damaged vouchers will not be replaced. 6. The merchant reserves the right to modify these terms and conditions at any time without prior notice",
    "variants": [
        {
            "variantID": "VAR-23c2a475-06cb-4118",
            "variantName": "TEST_PRODUCT_1_DP_APIS INR 100",
            "variantDisplayName": "₹ 100",
            "mrp": 100,
            "price": 99,
            "margin": 1,
            "stock": 999
        }
    ]
},
{
    "productID": "PROD-cf51c24f-425a-463b",
    "attachments": [
        "https://storage.googleapis.com/exlr8-assets/0e6ffdd-67e7-41f1-bf87-12adc597d4cb/images/products/default\_voucher.jpg"
    ]
}
```

```
],
  "categories": [
    {
      "categoryID": "CAT-e0bc01c8-a9b2-4d3a",
      "categoryName": "Gaming"
    }
  ],
  "descriptionText": "Get this voucher for instant savings on your next purchase. Simply redeem it at the time of checkout to apply the discount. It's the perfect way to make your money go further",
  "productDisplayName": "TEST_PRODUCT_2_DP_APIS",
  "productName": "TEST_PRODUCT_2_DP_APIS",
  "redemptionInstructions": "To redeem your voucher, follow these steps:\n\nClick on the unique redemption URL provided in your email or on the voucher page.\n\nAdd your desired products to the cart on the partner's website.\n\nThe discount will be automatically applied at checkout, or you may need to enter a unique code found on your voucher.\n\nComplete your purchase and enjoy your savings!",
  "termsAndConditions": "This voucher is valid for one-time use only. 2. It cannot be combined with any other offers, discounts, or promotions. 3. This voucher is non-refundable and cannot be exchanged for cash. 4. The voucher is valid until its stated expiration date. 5. Lost, stolen, or damaged vouchers will not be replaced. 6. The merchant reserves the right to modify these terms and conditions at any time without prior notice",
  "variants": [
    {
      "variantID": "VAR-3455a0e1-47c8-4e6f",
      "variantName": "TEST_PRODUCT_2_DP_APIS INR 100",
      "variantDisplayName": "₹ 100",
      "mrp": 100,
      "price": 99,
      "margin": 1,
      "stock": 3999
    }
  ]
},
  "paginationInfo": {
```

```
"nextCursor": "",
"hasMore": false
}
}
```

Response Fields

Product Fields

Field	Type	Description
productID	String	Unique product identifier
productName	String	Internal product name
productDisplayName	String	Display name for customers
descriptionText	String	Product description
redemptionInstructions	String	How to redeem the product
termsAndConditions	String	Terms and conditions
attachments	Array	Product images and assets
categories	Array	Product categories
variants	Array	Available product variants

Variant Fields

Field	Type	Description
variantID	String	Unique variant identifier
variantName	String	Internal variant name
variantDisplayName	String	Display name for customers
mrp	Number	Maximum retail price
price	Number	Your cost price
margin	Number	Margin percentage
stock	Number	Available stock quantity

Category Fields

Field	Type	Description
categoryID	String	Category identifier
categoryName	String	Category display name

Product Categories

Common product categories include:

- Entertainment & Gaming
- Software & Technology
- E-commerce & Retail

- Food & Beverage
- Travel & Transportation
- Telecommunications

Pagination Example

```
// Example: Fetch all products
async function getAllProducts() {
  let allProducts = [];
  let nextCursor = null;

  do {
    const url = nextCursor
      ? `${baseUrl}/products/delivery-
partners/${dpID}?nextCursor=${nextCursor}&limit=100`
      : `${baseUrl}/products/delivery-partners/${dpID}?limit=100`;

    const response = await fetch(url, {
      headers: {
        "x-client-id": clientId,
        "x-client-secret": clientSecret,
      },
    });

    const data = await response.json();
    allProducts.push(...data.products);
    nextCursor = data.paginationInfo.hasMore
      ? data.paginationInfo.nextCursor
      : null;
  } while (nextCursor);

  return allProducts;
}
```

Error Responses

Unauthorized Access

```
{  
  "error": "unauthenticated",  
  "errCode": "UNAUTHORIZED"  
}
```

Invalid DP ID

```
{  
  "error": "forbidden: param: admin user does not have access to DP:  
INVALID_DP_ID",  
  "errCode": "FORBIDDEN"  
}
```

Use Cases

1. Product Catalog Display

Show available products to your customers with pricing and descriptions.

2. Inventory Management

Track which products and variants are available for ordering.

3. Price Comparison

Compare pricing across different variants and products.

4. Category Browsing

Allow customers to browse products by category.

Best Practices

1. **Cache Product Data:** Products don't change frequently, so implement caching
2. **Use Pagination:** Fetch products in manageable chunks
3. **Handle Images:** Properly display product attachments/images
4. **Filter by Category:** Allow users to filter by product categories
5. **Show Availability:** Display real-time availability information

Integration Example

```
// Example: Build product catalog
async function buildProductCatalog() {
  const products = await getAllProducts();

  // Group by categories
  const catalog = {};

  products.forEach((product) => {
    product.categories.forEach((category) => {
      if (!catalog[category.categoryName]) {
        catalog[category.categoryName] = [];
      }
      catalog[category.categoryName].push({
        ...product,
        variants: product.variants.map((variant) => ({
          ...variant,
          displayPrice: `₹${variant.price}`,
          savings:
            variant.mrp > variant.price ? variant.mrp -
variant.price : 0,
        })),
      }));
    });
  });
}
```

```
    });  
    });  
});  
  
return catalog;  
}
```

Get Product by ID

Get detailed information about a specific product and all its variants.

Endpoint

GET /v1/products/delivery-partners/{dpID}/{productID}

Headers

Header	Type	Description	Required
x-client-id	String	Your clientID	☒ Yes
x-client-secret	String	Your clientSecret	☒ Yes

Path Parameters

Parameter	Type	Description	Required
-----------	------	-------------	----------

dpID	String	Your delivery partner ID	☒ Yes
productID	String	Product identifier	☒ Yes

PRODUCT ID FORMAT

Product IDs follow the pattern: PROD-{UUID} (e.g., PROD-cd250a2a-2b27-40e0)

Example Request

```
curl --location 'https://stage-platform-exlr8.exlr8now.com/v1/products/delivery-partners/YOUR_DP_ID/PROD-cd250a2a-2b27-40e0' \
--header 'x-client-id: YOUR_CLIENT_ID' \
--header 'x-client-secret: YOUR_CLIENT_SECRET'
```

Response

Successful Response

```
{
  "productID": "PROD-cd250a2a-2b27-40e0",
  "attachments": [
    "https://storage.googleapis.com/exlr8-assets/0e6fffdd-67e7-41f1-bf87-12adc597d4cb/images/products/default_voucher.jpg"
  ],
  "categories": [
    {
      "categoryID": "CAT-e0bc01c8-a9b2-4d3a",
      "categoryName": "Gaming"
    }
  ],
  "descriptionText": "Get this voucher for instant savings on your next purchase. Simply redeem it at the time of checkout to apply the
```

```
discount. It's the perfect way to make your money go further",
  "productDisplayName": "TEST_PRODUCT_1_DP_APIS",
  "productName": "TEST_PRODUCT_1_DP_APIS",
  "redemptionInstructions": "To redeem your voucher, follow these
steps:\n\nClick on the unique redemption URL provided in your email or
on the voucher page.\n\nAdd your desired products to the cart on the
partner's website.\n\nThe discount will be automatically applied at
checkout, or you may need to enter a unique code found on your
voucher.\n\nComplete your purchase and enjoy your savings!",
  "termsAndConditions": "This voucher is valid for one-time use only.
2. It cannot be combined with any other offers, discounts, or
promotions. 3. This voucher is non-refundable and cannot be exchanged
for cash. 4. The voucher is valid until its stated expiration date. 5.
Lost, stolen, or damaged vouchers will not be replaced. 6. The
merchant reserves the right to modify these terms and conditions at
any time without prior notice",
  "variants": [
    {
      "variantID": "VAR-23c2a475-06cb-4118",
      "variantName": "TEST_PRODUCT_1_DP_APIS INR 100",
      "variantDisplayName": "₹ 100",
      "mrp": 100,
      "price": 99,
      "margin": 1,
      "stock": 999
    }
  ]
}
```

Response Fields

Product Information

Field	Type	Description
-------	------	-------------

productID	String	Unique product identifier
productName	String	Internal product name
productDisplayName	String	Customer-facing display name
descriptionText	String	Detailed product description
redemptionInstructions	String	Step-by-step redemption guide
termsAndConditions	String	Product terms and conditions
attachments	Array	Product images and marketing materials
categories	Array	Product categories
variants	Array	All available variants for this product

Variant Information

Field	Type	Description
variantID	String	Unique variant identifier (use for ordering)
variantName	String	Internal variant name
variantDisplayName	String	Customer-facing variant name
mrp	Number	Maximum retail price
price	Number	Your wholesale/cost price
margin	Number	Margin percentage

stock	Number	Available stock quantity
-------	--------	--------------------------

Category Fields

Field	Type	Description
categoryID	String	Category identifier
categoryName	String	Category display name

Product Details Usage

This endpoint is ideal for:

1. Product Detail Pages

Show comprehensive product information to customers before purchase.

2. Variant Selection

Display all available denominations/variants for a product.

3. Pricing Information

Get current pricing for specific products.

Integration Example

```
// Example: Create product detail page
async function getProductDetails(productID) {
```

```

try {
  const response = await fetch(
    `${baseUrl}/products/delivery-partners/${dpID}/${productID}`,
    {
      headers: {
        "x-client-id": clientId,
        "x-client-secret": clientSecret,
      },
    }
  );

  if (!response.ok) {
    const error = await response.json();
    throw new Error(`${error.errCode}: ${error.error}`);
  }

  const product = await response.json();

  // Format for display
  return {
    ...product,
    formattedVariants: product.variants.map((variant) => ({
      ...variant,
      displayPrice: `₹${variant.price.toLocaleString()}`,
      originalPrice:
        variant.mrp > 0 ? `₹${variant.mrp.toLocaleString()}` : null,
      discount:
        variant.mrp > variant.price
          ? Math.round(((variant.mrp - variant.price) / variant.mrp)
            * 100)
          : 0,
    })),
    categoryNames: product.categories
      .map((cat) => cat.categoryName)
      .join(", "),
  };
} catch (error) {
  console.error("Error fetching product details:", error);
  throw error;
}

```

```
}  
}
```

Error Responses

Product Not Found

```
{  
  "error": "product not found for dpID: YOUR_DP_ID, productID:  
PRODUCT_ID",  
  "errCode": "RECORD_NOT_FOUND"  
}
```

Unauthorized Access

```
{  
  "error": "unauthenticated",  
  "errCode": "UNAUTHORIZED"  
}
```

Access Denied

```
{  
  "error": "forbidden: param: admin user does not have access to DP:  
INVALID_DP_ID",  
  "errCode": "FORBIDDEN"  
}
```

Best Practices

1. **Cache Product Details:** Product information changes infrequently
2. **Handle Missing Images:** Gracefully handle products without attachments
3. **Display Variants Clearly:** Show all available denominations/options
4. **Format Pricing:** Present pricing in user-friendly formats
5. **Show Redemption Info:** Clearly display how customers can use the product

Use Cases

1. Product Comparison

Compare different variants of the same product.

2. Customer Education

Show detailed product information and redemption instructions.

3. Inventory Planning

Understand available variants for inventory planning.

Place Order

Place a new order using a unique external reference and product variant.

Endpoint

```
POST /v1/orders/b2b/direct-checkout
```

Headers

Header	Type	Description	Required
x-client-id	String	Your clientID	☒ Yes
x-client-secret	String	Your clientSecret	☒ Yes
Content-Type	String	Must be application/json	☒ Yes

Request Body

For Product Orders

Field	Type	Description	Required
dpID	String	Your delivery partner ID (same as clientID)	☒ Yes
variantID	String	Product variant identifier	☒ Yes
externalRefID	String	Unique reference ID provided by client	☒ No

```
{
  "dpID": "{YOUR_DP_ID}",
  "variantID": "VAR-3455a0e1-47c8-4e6f",
  "externalRefID": "{uniqueRefIDToBeProvidedByClient}"
}
```

UNIQUE EXTERNAL REFERENCE

The externalRefID must be unique for each order. This is your reference for tracking the order in your system.

Example Request

```
curl --location 'https://stage-platform-exlr8.exlr8now.com/v1/orders/b2b/direct-checkout' \
--header 'x-client-id: YOUR_CLIENT_ID' \
--header 'x-client-secret: YOUR_CLIENT_SECRET' \
--header 'Content-Type: application/json' \
--data '{
  "dpID": "{YOUR_DP_ID}",
  "variantID": "VAR-3455a0e1-47c8-4e6f",
  "externalRefID": "order-07-09-2025"
}'
```

Response

Successful Response

```
{
  "orderId": "ORDIN080920252e9625e",
  "externalRefID": "order-07-09-2025",
  "dpUserEmail": "johndoe@gmail.com",
  "dpUserName": "Test DP",
  "dpID": "51d151ae-7d8d-4160-91dd-ef849746f789",
  "totalOrderMRP": 100,
  "totalAmount": 99,
  "totalCostPrice": 99,
  "status": "COMPLETED",
  "fulfillmentStatus": "FULFILLED",
  "createdAt": "2025-09-08T10:03:55.541Z",
  "updatedAt": "2025-09-08T10:04:50.879Z",
  "assetURL": "",
  "type": "DIRECT_CHECKOUT",
  "lineItems": [
    {
      "variantID": "VAR-3455a0e1-47c8-4e6f",
```

```

    "productID": "PROD-cf51c24f-425a-463b",
    "quantity": 1,
    "mrp": 100,
    "price": 99,
    "totalMRP": 100,
    "totalPrice": 99,
    "variantName": "TEST_PRODUCT_2_DP_APIS INR 100",
    "productName": "TEST_PRODUCT_2_DP_APIS",
    "variantDisplayName": "₹ 100",
    "productDisplayName": "TEST_PRODUCT_2_DP_APIS",
    "attachments": [
      "https://storage.googleapis.com/exlr8-
assets/voucher\_bulk\_uploads/default\_voucher.jpg"
    ],
    "mobileNumbers": null,
    "vouchers": [
      {
        "voucherCode": "78776667",
        "voucherPin": "T6R9-Q3M8-C4VVB1200",
        "expirationDate": "2028-01-02T00:00:00Z"
      }
    ],
    "fulfillmentStatus": "FULFILLED",
    "allocatedQty": 1,
    "fulfilledQty": 1
  }
]
}

```

Response Fields

Field	Type	Description
orderID	String	System-generated unique order identifier
externalRefID	String	Your provided external reference

dpUserEmail	String	Email of the DP user
dpUserName	String	Name of the DP user
dpID	String	Your delivery partner ID
totalOrderMRP	Number	Total MRP of the order
totalAmount	Number	Total amount charged
totalCostPrice	Number	Total cost price
status	String	Order status (see Status Codes)
fulfillmentStatus	String	Fulfillment status (see Status Codes)
createdAt	String	Order creation timestamp (ISO 8601)
updatedAt	String	Last update timestamp (ISO 8601)
assetURL	String	URL to download order details (when fulfilled)
type	String	Order type
lineItems	Array	Array of ordered items with details

Line Item Fields

Field	Type	Description
variantID	String	Product variant identifier
productID	String	Product identifier

quantity	Number	Quantity ordered
mrp	Number	Maximum retail price per unit
price	Number	Actual price charged per unit
totalMRP	Number	Total maximum retail price for the line item
totalPrice	Number	Total actual price charged for the line item
variantName	String	Variant name
productName	String	Product name
variantDisplayName	String	Display name for the variant
productDisplayName	String	Display name for the product
attachments	Array	Array of attachment URLs (e.g., images or documents)
mobileNumbers	Array	Array of mobile numbers (if applicable, e.g., for SIM cards)
vouchers	Array	Voucher details (when fulfilled) - see Voucher Fields
fulfillmentStatus	String	Item fulfillment status (see Status Codes)
allocatedQty	Number	Quantity allocated
fulfilledQty	Number	Quantity fulfilled

Voucher Fields

When vouchers are available (order status is `COMPLETED` and fulfillment status is `FULFILLED`), each voucher object contains:

Field	Type	Description
voucherCode	String	The voucher code to be used by the customer
voucherPin	String	The voucher PIN for activation
expirationDate	String	Voucher expiration date (ISO 8601 format)

Example voucher object:

```
{
  "voucherCode": "78776667",
  "voucherPin": "T6R9-Q3M8-C4VVB1200",
  "expirationDate": "2028-01-02T00:00:00Z"
}
```

Order Status Codes

The order will have one of two final statuses:

Status	Description	Next Steps
COMPLETED	Order successfully completed	Download vouchers if available
FAILED	Order failed during processing	Check error details and retry

Asset Download

When order status is COMPLETED and fulfillmentStatus is FULFILLED:

1. **Download the file** from assetURL.
2. **Use the password** sent over email to open the file.
3. **Extract voucher information** for delivery to your customers.

Error Responses

Insufficient Balance

```
{  
  "error": "insufficient wallet balance",  
  "errCode": "INSUFFICIENT_BALANCE"  
}
```

Product Not Available

```
{  
  "error": "insufficient stock for variant VARIANT_ID. Available:  
AVAILABLE_STOCK_COUNT, Requested: REQUESTED_STOCK_COUNT",  
  "errCode": "OUT_OF_STOCK"  
}
```

Invalid Request

```
{  
  "error": "invalid variant ID",  
  "errCode": "BAD_REQUEST"  
}
```

Best Practices

1. **Unique External References:** Always use unique externalRefID values
2. **Error Handling:** Implement proper error handling for all response codes
3. **Status Check:** Check order status to see if it's COMPLETED or FAILED
4. **Asset Download:** Download and securely store voucher files when status is COMPLETED

Get Orders

Get a paginated list of orders with optional filters such as date range and pagination parameters.

Endpoint

```
GET /v1/orders/b2b/delivery-partners/{dpID}
```

Headers

Header	Type	Description	Required
x-client-id	String	Your clientID	☒ Yes
x-client-secret	String	Your clientSecret	☒ Yes

Path Parameters

Parameter	Type	Description	Required
dpID	String	Your delivery partner ID	☒ Yes

Query Parameters

Parameter	Type	Description	Required	Default
nextCursor	String	Pagination cursor for next page	✗ No	-
limit	Integer	Number of records per page	✗ No	15

PAGINATION LIMITS

- **Default limit:** 15 records per page
- **Maximum limit:** 100 records per page

Example Request

Get First Page of Orders

```
curl --location 'https://stage-platform-exlr8.exlr8now.com/v1/orders/b2b/delivery-partners/YOUR_DP_ID' \  
  --header 'x-client-id: YOUR_CLIENT_ID' \  
  --header 'x-client-secret: YOUR_CLIENT_SECRET'
```

Get Orders with Pagination

```
curl --location 'https://stage-platform-exlr8.exlr8now.com/v1/orders/b2b/delivery-partners/YOUR_DP_ID' \  
  --data-urlencode 'limit={limit}' \  
  --data-urlencode 'nextCursor={nextCursor}' \  
  --header 'x-client-id: YOUR_CLIENT_ID' \  
  --header 'x-client-secret: YOUR_CLIENT_SECRET'
```

Response

Successful Response

```
{
  "orders": [
    {
      "orderID": "ORDIN080920252e9625e",
      "externalRefID": "order-07-09-2025",
      "dpUserEmail": "johndoe@gmail.com",
      "dpUserName": "Test DP",
      "dpID": "51d151ae-7d8d-4160-91dd-ef849746f789",
      "totalOrderMRP": 100,
      "totalAmount": 99,
      "totalCostPrice": 99,
      "status": "COMPLETED",
      "fulfillmentStatus": "FULFILLED",
      "createdAt": "2025-09-08T10:03:55.541Z",
      "updatedAt": "2025-09-08T10:04:50.879Z",
      "assetURL": "",
      "type": "DIRECT_CHECKOUT",
      "lineItems": [
        {
          "variantID": "VAR-3455a0e1-47c8-4e6f",
          "productID": "PROD-cf51c24f-425a-463b",
          "quantity": 1,
          "mrp": 100,
          "price": 99,
          "totalMRP": 100,
          "totalPrice": 99,
          "variantName": "TEST_PRODUCT_2_DP_APIS INR 100",
          "productName": "TEST_PRODUCT_2_DP_APIS",
          "variantDisplayName": "₹ 100",
          "productDisplayName": "TEST_PRODUCT_2_DP_APIS",
          "attachments": [
            "https://storage.googleapis.com/exlr8-
            assets/voucher_bulk_uploads/default_voucher.jpg"
          ],
        }
      ],
    }
  ],
}
```

```

    "mobileNumbers": null,
    "vouchers": [
      {
        "voucherCode": "78776667",
        "voucherPin": "T6R9-Q3M8-C4VVB1200",
        "expirationDate": "2028-01-02T00:00:00Z"
      }
    ],
    "fulfillmentStatus": "FULFILLED",
    "allocatedQty": 1,
    "fulfilledQty": 1
  }
]
}
],
"paginationInfo": {
  "nextCursor": "",
  "hasMore": false
}
}

```

Response Fields

Field	Type	Description
orders	Array	Array of order objects
paginationInfo	Object	Pagination information

Pagination Info

Field	Type	Description
nextCursor	String	Cursor for the next page (empty if no more pages)

hasMore	Boolean	Whether more records are available
---------	---------	------------------------------------

Pagination Example

```
// Example: Fetch all orders
let allOrders = [];
let nextCursor = null;

do {
  const url = nextCursor
    ? `${baseUrl}/orders/b2b/delivery-
partners/${dpID}?nextCursor=${nextCursor}`
    : `${baseUrl}/orders/b2b/delivery-partners/${dpID}`;

  const response = await fetch(url, {
    headers: {
      "x-client-id": clientId,
      "x-client-secret": clientSecret,
    },
  });

  const data = await response.json();
  allOrders.push(...data.orders);
  nextCursor = data.paginationInfo.hasMore
    ? data.paginationInfo.nextCursor
    : null;
} while (nextCursor);
```

Error Responses

Unauthorized Access

```
{  
  "error": "unauthenticated",  
  "errCode": "UNAUTHORIZED"  
}
```

Invalid DP ID

```
{  
  "error": "forbidden: param: admin user does not have access to DP:  
INVALID_DP_ID",  
  "errCode": "FORBIDDEN"  
}
```

Use Cases

1. Order History Dashboard

Display recent orders with pagination for your users.

2. Reconciliation

Fetch all orders for a specific time period to reconcile with your records.

3. Status Monitoring

Regularly poll for orders that need status updates.

Best Practices

1. **Use Pagination:** Don't try to fetch all orders at once
2. **Implement Caching:** Cache order data to reduce API calls
3. **Handle Empty Results:** Gracefully handle cases with no orders
4. **Monitor Performance:** Use appropriate page sizes for your use case

Get Order by ID

Fetch detailed information for a specific order using the system-generated order ID.

Endpoint

```
GET /v1/orders/b2b/delivery-partners/{dpID}/{orderID}
```

Headers

Header	Type	Description	Required
x-client-id	String	Your clientID	☒ Yes
x-client-secret	String	Your clientSecret	☒ Yes

Path Parameters

Parameter	Type	Description	Required
-----------	------	-------------	----------

dpID	String	Your delivery partner ID	☒ Yes
orderID	String	System-generated order identifier	☒ Yes

ORDER ID FORMAT

Order IDs follow the pattern: ORDIN{DATE}{RANDOM} (e.g., ORDIN080920252e9625e)

Example Request

```
curl --location 'https://stage-platform-exlr8.exlr8now.com/v1/orders/b2b/delivery-partners/YOUR_DP_ID/ORDIN080920252e9625e' \
--header 'x-client-id: YOUR_CLIENT_ID' \
--header 'x-client-secret: YOUR_CLIENT_SECRET'
```

Response

Successful Response

```
{
  "orderID": "ORDIN080920252e9625e",
  "externalRefID": "order-07-09-2025",
  "dpUserEmail": "johndoe@gmail.com",
  "dpUserName": "Test DP",
  "dpID": "51d151ae-7d8d-4160-91dd-ef849746f789",
  "totalOrderMRP": 100,
  "totalAmount": 99,
  "totalCostPrice": 99,
  "status": "COMPLETED",
  "fulfillmentStatus": "FULFILLED",
  "createdAt": "2025-09-08T10:03:55.541Z",
  "updatedAt": "2025-09-08T10:04:50.879Z",
  "assetURL": "",
}
```

```

"type": "DIRECT_CHECKOUT",
"lineItems": [
  {
    "variantID": "VAR-3455a0e1-47c8-4e6f",
    "productID": "PROD-cf51c24f-425a-463b",
    "quantity": 1,
    "mrp": 100,
    "price": 99,
    "totalMRP": 100,
    "totalPrice": 99,
    "variantName": "TEST_PRODUCT_2_DP_APIS INR 100",
    "productName": "TEST_PRODUCT_2_DP_APIS",
    "variantDisplayName": "₹ 100",
    "productDisplayName": "TEST_PRODUCT_2_DP_APIS",
    "attachments": [
      "https://storage.googleapis.com/exlr8-
assets/voucher bulk uploads/default voucher.jpg"
    ],
    "mobileNumbers": null,
    "vouchers": [
      {
        "voucherCode": "78776667",
        "voucherPin": "T6R9-Q3M8-C4VVB1200",
        "expirationDate": "2028-01-02T00:00:00Z"
      }
    ],
    "fulfillmentStatus": "FULFILLED",
    "allocatedQty": 1,
    "fulfilledQty": 1
  }
]
}

```

Example: Download order assets

```

curl --location 'https://storage.googleapis.com/exlr8-
assets/orders/ORDIN080920252e9625e_order_details.xlsx' \
--output order_details.xlsx

```

Error Responses

Order Not Found

```
{  
  "error": "order not found with orderID: INVALID_ORDER_ID and dpId:  
YOUR_DP_ID",  
  "errCode": "RECORD_NOT_FOUND"  
}
```

Use Cases

1. Order Status Updates

Regularly check order status to provide updates to your customers.

2. Voucher Retrieval

Download and process voucher information once orders are fulfilled.

3. Customer Support

Provide detailed order information for customer inquiries.

Best Practices

1. **Poll Wisely:** Don't poll too frequently for status updates
2. **Cache Results:** Cache order details to reduce API calls
3. **Handle All States:** Implement logic for all possible order states
4. **Secure Asset Handling:** Properly handle and store downloaded voucher files

Get Order by External Reference

Fetch detailed information for a specific order using your provided external reference ID.

Endpoint

```
GET /v1/orders/b2b/delivery-partners/{dpID}/externalref/{externalRefID}
```

Headers

Header	Type	Description	Required
x-client-id	String	Your clientID	☒ Yes
x-client-secret	String	Your clientSecret	☒ Yes

Path Parameters

Parameter	Type	Description	Required
dpID	String	Your delivery partner ID	☒ Yes

externalRefID	String	Your unique external reference ID	☒ Yes
---------------	--------	-----------------------------------	---

USE YOUR REFERENCE

This endpoint is perfect when you want to track orders using your own reference system instead of eXlr8's order IDs.

Example Request

```
curl --location 'https://stage-platform-exlr8.exlr8now.com/v1/orders/b2b/delivery-partners/YOUR_DP_ID/externalref/order-07-09-2025' \
--header 'x-client-id: YOUR_CLIENT_ID' \
--header 'x-client-secret: YOUR_CLIENT_SECRET'
```

Response

Successful Response

```
{
  "orderId": "ORDIN080920252e9625e",
  "externalRefID": "order-07-09-2025",
  "dpUserEmail": "johndoe@gmail.com",
  "dpUserName": "Test DP",
  "dpID": "51d151ae-7d8d-4160-91dd-ef849746f789",
  "totalOrderMRP": 100,
  "totalAmount": 99,
  "totalCostPrice": 99,
  "status": "COMPLETED",
  "fulfillmentStatus": "FULFILLED",
  "createdAt": "2025-09-08T10:03:55.541Z",
  "updatedAt": "2025-09-08T10:04:50.879Z",
  "assetURL": "",
  "type": "DIRECT_CHECKOUT",
  "lineItems": [
```

```

{
  "variantID": "VAR-3455a0e1-47c8-4e6f",
  "productID": "PROD-cf51c24f-425a-463b",
  "quantity": 1,
  "mrp": 100,
  "price": 99,
  "totalMRP": 100,
  "totalPrice": 99,
  "variantName": "TEST_PRODUCT_2_DP_APIS INR 100",
  "productName": "TEST_PRODUCT_2_DP_APIS",
  "variantDisplayName": "₹ 100",
  "productDisplayName": "TEST_PRODUCT_2_DP_APIS",
  "attachments": [
    "https://storage.googleapis.com/exlr8-
assets/voucher bulk uploads/default_voucher.jpg"
  ],
  "mobileNumbers": null,
  "vouchers": [
    {
      "voucherCode": "78776667",
      "voucherPin": "T6R9-Q3M8-C4VVB1200",
      "expirationDate": "2028-01-02T00:00:00Z"
    }
  ],
  "fulfillmentStatus": "FULFILLED",
  "allocatedQty": 1,
  "fulfilledQty": 1
}
]
}

```

Response Schema

The response schema is identical to [Get Order by ID](#). See that page for detailed field descriptions.

Error Responses

Order Not Found

```
{  
  "error": "order not found for external reference: order-07-09-2025",  
  "errCode": "RECORD_NOT_FOUND"  
}
```

Use Cases

1. Customer Service Integration

Quickly look up orders when customers provide their reference numbers.

2. Internal System Synchronization

Sync order status with your internal systems using your own reference IDs.

3. Webhook Processing

Update order status in your system when receiving webhooks with external references.

Integration Example

```
// Example: Check order status by external reference  
async function checkOrderStatus(externalRefID) {  
  try {  
    const response = await fetch(  
      `${baseUrl}/orders/b2b/delivery-  
partners/${dpID}/externalref/${externalRefID}`,  
    );
```

```

    {
      headers: {
        "x-client-id": clientId,
        "x-client-secret": clientSecret,
      },
    }
  );

  if (!response.ok) {
    const error = await response.json();
    if (error.errCode === "RECORD_NOT_FOUND") {
      console.log("Order not found");
      return null;
    }
    throw new Error(`API Error: ${error.error}`);
  }

  const order = await response.json();

  // Handle different order states
  switch (order.status) {
    case "COMPLETED":
      if (order.fulfillmentStatus === "FULFILLED") {
        await downloadOrderAssets(order);
      }
      break;
    case "FAILED":
      await handleFailedOrder(order);
      break;
    case "PROCESSING":
      // Schedule another check later
      setTimeout(() => checkOrderStatus(externalRefID), 30000);
      break;
  }

  return order;
} catch (error) {
  console.error("Error checking order status:", error);
  throw error;
}

```

```
}  
}
```

Best Practices

1. **Consistent References:** Use a consistent format for your external reference IDs
2. **Error Handling:** Always handle the case where orders are not found
3. **Status Polling:** Implement intelligent polling for pending orders
4. **Logging:** Log all order lookups for audit trails